

SQUIGGLE

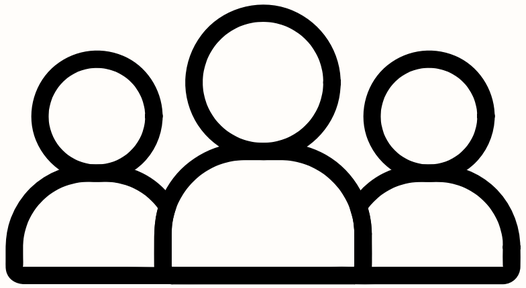
Sketch.Think.Guess

A REAL-TIME DRAWING & GUESSING GAME



Meet the Team

TEAM 7



ASHFAQUE SADAD 230042102	**SHEIKH ALIMUN ALAHI** 230042115	**MD ZAHIN UDDIN NIROB** 230042160

PROJECT OVERVIEW



Real-time multiplayer drawing & guessing game



Focus on creativity, fun and social interaction



Inspired by games like Skribbl with enhanced features



Accessible and engaging for all types of players

CORE CLASSES & ENTITY DESCRIPTIONS

User

Stores player login info, coin balances, and game stats like wins and streaks.

`goldCoins` `diamonds` `totalWins`

Avatar

The player's visual look — body shape, color, and accessory. Lives inside User.

`bodyType` `hexColor` `accessory`

Room

A game room saved in the database with a join code, host info, and current status.

`roomId` `host` `MongoDB`

RoomSettings

Game options like max players, draw time, rounds, word categories, and privacy.

`maxPlayers` `countdowns` `genres`

GameSession

Saves the final results of a completed game — players, winner, and scores

`matchSummary` `finalResults`

SessionPlayer

A player's score and coins earned in a single game

`scores` `matchRewards`

Reward

Tracks coin and diamond transactions — rewards, streaks, and purchases.

`milestones` `streakBonus`

RoomState

Live game data kept in server memory — players, secret word, timer, hints, and scores.

`in-memory` `secretWords` `hints`

PROJECT STRUCTURE & MODULE ORGANIZATION

Project File Tree

```
Squiggle/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── context/
│   │   └── styles/
│   └── vite.config.js
├── server/
│   ├── routes/
│   ├── socket/
│   ├── models/
│   ├── services/
│   ├── utils/
│   └── index.js
└── package.json
```

FRONTEND — REACT

Components and pages manage user interaction, canvas, avatar, and gameplay UI.

BACKEND — NODE.JS

REST routes, Socket.IO events, game rules, and AI integration.

DATA — MONGODB

Mongoose models persist users, rooms, sessions, and rewards.

- **WHY THIS STRUCTURE?**

SEPARATION OF CONCERNS • REUSABILITY • MAINTAINABILITY • PARALLEL TEAMWORK



COMPONENT COMMUNICATION & DATA EXCHANGE

SYSTEM COMMUNICATION

React Frontend
Pages · Canvas · Lobby · Voice

REST + Socket.IO

Node.js / Express
Routes · Events · Game Rules

MongoDB
Mongoose

Gemini AI
Word Service

WebRTC Voice
Peer ↔ Peer audio, Socket.IO signaling

REST API — Authentication
POST / api / auth / login
Response: { token, user }

SOCKET.IO — Real-Time Guess
guessWord
{ roomId, guess }

MONGOOSE — Persistent Data
User: goldCoins, totalWins, avatar
Room, GameSession, Reward

INDIVIDUAL CONTRIBUTION

Ashfaq (102)

GAME ENGINE

- Round lifecycle and scoring
- Power-ups and rewards
- Vote-kick and moderation
- Game State · Economy · Rules

Alahi (115)

IDENTITY & VOICE

- Authentication and profiles
- Avatar and leaderboard
- Lobby and WebRTC voice

Zahin (160)

DRAWING & AI

- Canvas tools, shapes and flood-fill
- Wordle logic and interface
- Gemini AI word generation
- Canvas · Wordle · AI

CLASSES & METHODS

How the Code Is Divided

- React components manage UI and interactions
- Express routes handle API requests
- Mongoose models manage database records
- Socket.IO handlers manage real-time events
- Custom hooks handle reusable logic

Why This Structure?

- Separates frontend and backend responsibilities
- Makes components reusable and maintainable
- Keeps authentication and validation organized
- Supports real-time multiplayer communication

Code Size

- Source files: 46 JS/JSX files
- Total source: 12,618 lines
- Functions/methods: 672
- Average function: 25.4 lines
- Maximum function: 1,606 lines

DEMONSTRATION

THANK YOU

We are open for questions

GitHub Repository – Squiggle